Flymedia Next.js POS & Online Ordering Platform Documentation

Welcome to the official developer documentation for the **Flymedia Next.js POS & Online Ordering Platform**. This document describes the application modules, database schemas, local setup processes, and build commands. ---

1. Core Modules

A. Merchant Dashboard & Operations

***Route**:* `/dashboard` **Features**: *Category and menu management (adding items, modifiers, pricing)*. Table management (generating table numbers and QR codes). *Custom dynamic layouts (configuring background colors and uploading images for menu, checkout, and bookings)*. Manage Delivery Zones (`RADIAL DISTANCE` and `ZIPCODE` rules).

B. Diner Online Ordering Menu

***Route**:* `/order-online/[orgSlug]/menu` **Features**: *Browsing menus, selecting dine-in (via QR table codes), pickup, or delivery.* Live delivery availability check (address geocoding validation matching active zones). *Cart checkout and Stripe integration for card transactions.*

C. Diner Account Portal

Route: `/order-online/[orgSlug]/customer/profile` ***Features**:* Sidebar navigation (desktop) and mobile drawer drawer overlay (mobile). *Edit profile details (Name, Phone, Email)*. Standalone secure password modification form (validating current, new, and confirm passwords). *Interactive Leaflet Map for pinning shipping addresses, using Nominatim reverse-geocoding.* Loyalty points, order logs, and bill payment history.

D. POS Simulator & Kitchen Display System (KDS)

***Route**: `/pos` & `/kds` **Features**: *Kitchen cooking status trackers.* Live status push via WebSockets (Socket.io). ---

2. Technical Stack

***Framework**: Next.js 16.2.4 (App Router & Turbopack)* **Database & ORM**: MySQL database managed via Sequelize ***Live Notifications**: Socket.io Server & Client integrations* **Payments**: Stripe Checkout & Payment Intents ***Hybrid Wrapper**: Capacitor wrapping web resources for iOS/Android packaging ---*

3. Database Schema Overview

The primary tables in the system include: `organizations` : Stores merchants/entities owning restaurants. `stores` *: Stores restaurant physical settings, address coordinates, and active theme variables.* `customers` : Stores diner records, address books, loyalty balances, and password hashes. `orders` *: Logs dine-in, delivery, or pickup orders.* `delivery_zones` : Logs boundaries defined as `RADIAL DISTANCE` or `ZIPCODE` lists.

- `delivery_rules` : Configures delivery charges, sequences, and minimum orders.

---

4. Development Operations Guide

A. Environment Configuration

Create a `.env` file in the root folder with: ``` `env DB_HOST=127.0.0.1 DB_PORT=3306 DB_USER=root DB_PASSWORD=your_password DB_NAME=flymedia_db NEXTAUTH_SECRET=your_nextauth_secret NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_... STRIPE_SECRET_KEY=sk_test_...` ```

B. Command Reference

`ActionCommandDescription **Start Dev Server**` npm run dev `Runs the Next.js Turbopack development backend. **Build Assets**` npm run build `Compiles an optimized production build. **Seed Zones**` npx tsx scripts/seed-delivery-zones.ts `Feeds test radial (10km) and zipcode zones into the database.`

Capacitor Sync npx cap sync Synchronizes React pages and layout assets to native Android and iOS folders. ---

5. Capacitor Mobile Builds

Whenever you update frontend views or styles: 1. Re-compile the React bundle: `bash npm run build` 2. Propagate updates to native modules: `bash npx cap sync` 3. Open native projects inside Android Studio or Xcode to compile final .apk or .ipa app packages: `bash npx cap open android npx cap open ios``